

HelixConstitution

HelixConstitution

Универсальная инженерная конституция, наследуемая каждым проектом — антиблефовый закон, применяемый механически, распространяемый как единый Git-сабмодуль.

Краткое описание

HelixConstitution — это единый, независимый от проекта свод правил, подключаемый в виде Git-сабмодуля каждым проектом Helix/vasic-digital. Он кодифицирует неукоснительную инженерную дисциплину (антиблеф, валидация только на основе доказательств, безопасность данных и хоста, документация и покрытие тестами) и распространяет её на парк из 140+ репозиториях. Это основа управления, обеспечивающая согласованность всей экосистемы.

Краткая характеристика

Универсальный наследуемый Constitution, поставляемый в виде Git-сабмодуля. Определяет обязательные, не подлежащие обсуждению правила — антиблефовые проверки доказательств, защиту от ложноположительных результатов, безопасность данных и хоста, дисциплину в области покрытия тестами и документации, — которые каждый использующий проект наследует автоматически и может расширять, но не ослаблять.

Подробное описание

HelixConstitution — это канонический, единый источник истины для инженерных практик, разделяемых всеми проектами, которые подключают его в виде Git-сабмодуля. Инженерный закон распространяется и версионизируется точно так же, как код. Его центральный элемент — Constitution.md —

это документ объёмом около 1 МБ, непрерывно обновляемый и содержащий пронумерованные положения (семейство заветов §11.4.x, на данный момент до §11.4.170), а также руководства для конкретных агентов (CLAUDE.md, AGENTS.md, QWEN.md, GEMINI.md), которые импортируют его по ссылке. Таким образом, и люди, и каждый агент CLI работают по единому своду правил. Наследование организовано в три уровня: универсальная база (этот сабмодуль), проектный уровень (собственные Constitution/CLAUDE/AGENTS проекта, расширяющие базу) и необязательный уровень для подкаталогов. Правила применяются сверху вниз, причём проект может *ужесточать* их, но архитектурно лишён возможности *ослаблять*. В результате парк из 140+ репозиторий не может незаметно разойтись, поскольку разделяемая ими дисциплина зафиксирована, а не хранится в памяти.

Документ категорически независим от предметной области: любые упоминания конкретных вендоров, аппаратных SKU, портов или версий библиотек должны быть вынесены в собственный Constitution проекта, а универсальность никогда не предполагается — она должна быть *доказана* по четырёхступенчатому тесту, прежде чем правило будет допущено в базу. Философская основа документа — антиблеф, выраженный в виде взаимосвязанного семейства заветов: §1.1 защита от ложноположительных результатов, §11.4 завет качества для конечного пользователя, §11.4.6 запрет на догадки, §11.4.69 таксономия позитивных доказательств. Их совокупный эффект — одна жёсткая линия: планка для релиза никогда не определяется как «тесты прошли», а как «реальный пользователь может воспользоваться функцией», причём каждое успешное выполнение должно ссылаться на зафиксированные физические доказательства, иначе оно не засчитывается. Сопроводительный submodules-catalogue.md (142 репозитория) превращает вопрос «есть ли у нас уже что-то, что это делает?» в рефлекс «сначала проверь каталог, не реализуй заново». Вспомогательные скрипты находят сабмодуль на любой глубине вложенности и рассылают каждый коммит четырём независимым Git-провайдерам, так что единственный авторитетный свод правил невозможно потерять.

Почему мы это создали

Несколько крупных продуктовых приложений и десятки разрозненных переиспользуемых подмодулей, написанных одним владельцем, раз за разом заново выводили одни и те же выстраданные правила — и раз за разом сталкивались с одним и тем же классом ошибок: тесты и отчёты о статусе, которые утверждают, что всё в порядке, в то время как функция сломана для конечного пользователя («ложные срабатывания»

и «ложные провалы»). Каждая криминалистическая зацепка в Constitution фиксирует реальный инцидент (например, ложное срабатывание аудио-маршрутизации D3 от 20.05.2026, когда валидация прошла успешно при пустом поле «Используемый кодек», или гигантская кнопка в интерфейсе от 25.06.2026, прошедшая тесты на равенство токенов, хотя реальный экран был сломан). Constitution существует для того, чтобы раз и навсегда механически исключить весь этот класс нечестных успехов — чтобы дисциплина не размывалась между проектами и не забывалась потихоньку.

Почему это меняет правила игры

Это превращает инженерную культуру из «документации, которую надеются соблюдать» в унаследованный, версионированный и механически принудительный закон — разницу между руководством по стилю и компилятором. Одно обновление подмодуля одномоментно и с возможностью отслеживания обновляет правила для всего парка продуктов. Единственный антиблефовый завет *гарантированно* присутствует в каждом использующем репозитории не на основе доверия, а по построению: шлюз распространения буквально ищет номер пункта по всему парку, а парный мутационный тест доказывает, что сам шлюз не блефует — так что даже принуждение принуждается. Управление перестаёт быть мечтой на вики-странице, которую никто не читает, и становится проверяемым, тестируемым фактом, на который можно нацелить задачу CI.

Что здесь нового

- **Constitution как подмодуль** — инженерный закон распространяется и версионизируется точно так же, как код, с осознанными тегами вроде v1.0.0 и привязкой к проектам, так что каждый репозиторий *точно знает*, какую версию закона он использует.
- **Антиблеф как полноценная криминалистическая доктрина** — каждый пункт прослеживается до дословного распоряжения оператора и, зачастую, до конкретного реального инцидента, который его мотивировал, так что свод правил читается как прецедентное право, а не как чьё-то мнение.
- **Метатестирование самих правил (§1.1)** — каждый шлюз сопровождается мутацией, которая должна переводить тест из состояния PASS в FAIL, так что утверждение «шлюз не фиктивный» не декларируется, а доказывается при каждом запуске; шлюз, который никогда не падает, считается хуже, чем отсутствие шлюза вообще.

- **Заработанная универсальность** — явный четырёхчастный тест определяет, действительно ли правило универсально или лишь специфично для проекта, сохраняя базу компактной, переносимой и свободной от утечек вендорных зависимостей.

Как это используется во всех продуктах (какие возможности даёт)

Как **обязательный столп управления**, HelixConstitution — это не документ, к которому обращается команда, а несущая конструкция, на которой эта команда строится:

- **Каркас управления:** каждый проект Helix/vasic-digital добавляет его как подмодуль и импортирует из CLAUDE.md / AGENTS.md / QWEN.md или собственного Constitution.md; правила применяются безоговорочно, с первого коммита, без возможности отказа на уровне проекта.
- **Шлюзы и предписания:** он определяет четырёхслойную модель покрытия — наличие исходников, выживаемость при сборке, поведение в рантайме, отсутствие блефа у шлюза — которую функция должна пройти на всех четырёх уровнях, прежде чем будет считаться готовой, а также растущий список именованных предписаний: обработка учётных данных (§11.4.10), синхронизация документации (§11.4.60), мандат на подмодули контейнеров (§11.4.76), CodeGraph (§11.4.78), обязательное покрытие типами тестов (§11.4.169) и другие.
- **Распространение:** шлюзы CM-COVENANT-114-NNN-PROPAGATION проверяют наличие *дословного* текста пункта во всём парке потребляющих репозиторийев, так что завет не может быть тихо выброшен в каком-то углу системы; несоблюдение становится жёстким блокиратором релиза без лазеек для обхода.
- **Поиск:** submodules-catalogue.md превращает вопрос «есть ли у нас уже что-то, что делает X?» в ответ с одного взгляда ещё до того, как будет создан новый модуль, убивая дублирование усилий на корню.
- **Единство агентов AI:** один и тот же закон формулируется одинаково для всех агентов CLI (Claude Code, Codex/Cursor/Aider/OpenCode/Crush/Kimi через AGENTS.md, Qwen Code через QWEN.md), так что независимо от того, какой инструмент касается кода, он подчиняется одному и тому же завету.

Крупнейшие технические вызовы и способы их решения

- **Поиск субмодуля на произвольной глубине вложенности** — правило, погребённое на три уровня вглубь, всё равно должно находить закон, не зная, где он находится → `find_constitution.sh` поднимается по родительским директориям и рекурсивно следует по указателю суперпроекта Git, учитывая переопределение `CONSTITUTION_DIR` и две поддерживаемые структуры (`constitution/`, `submodules/constitution/`), благодаря чему разрешение остаётся детерминированным вне зависимости от глубины вложенности.
- **Поддержка единого авторитетного репозитория на четырёх Git-провайдерах** — зеркала бесполезны, если они расходятся → `install_upstreams.sh` считывает декларативные удалённые репозитории из `Upstreams/*.sh` и настраивает `origin` с несколькими URL для push, так что одна команда `git push` атомарно рассылается на GitHub (основной), GitLab, GitFlic и GitVerse, и ни одно зеркало не отстаёт.
- **Предотвращение разрастания правил и утечки проектных зависимостей в универсальную базу** — каждое соблазнительное «просто добавь это сюда» подрывает переносимость → к каждому новому правилу применяется четырёхэтапный тест на «заслуженную универсальность» и классификация по §11.4.17 (универсальное vs. проектное), что заставляет проектные особенности оставаться на уровне проекта, где им и место.
- **Проверка работоспособности механизма наследования** — ворота, которые никогда не дают сбоя, — это ворота, которым нельзя доверять → `meta_test_inheritance.sh`, сторожевой мета-тест, намеренно удаляет якорь §11.4 и проверяет, что механизм его перехватывает, так что сам механизм принудительно перепроверяется на предмет скрытых поломок.

Технологический стек

- **Наследование через Git-субмодули** — *почему*: Git-субмодули — единственный механизм, позволяющий своду правил быть авторитетным и привязанным к версии для каждого потребителя, обновляемым через явный, рецензируемый коммит, а не через молчаливое копирование; *как*: проекты-потребители подключают субмодуль и импортируют его агентские файлы через `@import`, а три слоя интерпретируются сверху вниз с жёстким контрактом «расширяет, а не ослабляет» на каждом уровне.

- **find_constitution.sh** — *почему*: правила бесполезны, если глубоко вложенный код не может их надёжно найти, а жёстко прописанные пути сломаются при первой же реорганизации проекта; *как*: подъём по родительским директориям с рекурсией через `git rev-parse --show-superproject-working-tree`, подстрахованный переопределением `CONSTITUTION_DIR`, что позволяет разрешать обе поддерживаемые структуры.
- **install_upstreams.sh + Upstreams/** — *почему*: четырёхпровайдерная избыточность реальна только если не требует дополнительных усилий для поддержки, иначе зеркала приходят в негодность; *как*: декларативные .sh-файлы для каждого удалённого репозитория материализуются в единый origin с несколькими URL, сводя четыре push-операции в одну.
- **Мета-тесты мутаций §1.1** — *почему*: ворота, которые никогда не дают сбоя, хуже их отсутствия, так как создают ложную уверенность; *как*: каждое ворота сопряжено с мутацией (удаление/переименование через sed), которая должна переводить тест из PASS в FAIL, после чего изменения откатываются, так что каждое ворота доказывает свою работоспособность при каждом запуске.
- **Ворота распространения (CM-COVENANT-114-NNN-PROPAGATION)** — *почему*: соглашение универсально только если оно проверяемо присутствует в *каждом* потребителе, а не только в флагманском репозитории; *как*: буквальный поиск по номеру пункта в коде потребителей, подкреплённый парным мета-тестом §1.1, который доказывает, что проверка распространения сама может давать сбой.
- **submodules-catalogue.md (§11.4.74)** — *почему*: самый быстрый способ нарушить дисциплину антидублирования — не знать, чем уже владеешь; *как*: каталог из 142 репозиторияев, сгруппированных по возможностям, с проверкой каталога в трекере до создания чего-либо нового.
- **Экспорт в несколько форматов** — *почему*: один и тот же закон должен быть одинаково доступен для чтения человеком, парсинга инструментами и архивирования; *как*: каждый канонический документ выпускается в форматах .md / .html / .pdf / .docx из одного источника.

Заметки о статусе и добросовестности

- **Статус: отправлено.** Активно поддерживается и используется в качестве субмодуля в рамках всего парка (публичные канонические и зеркальные репозитории).
- **Лицензия: уточняется** — в проанализированных исходных материалах не указана; перед публикацией подтвердить в файле LICENSE репозитория.

- Дополнительные зеркала вышестоящих репозиторий:
GitLab helixdevelopment1/helixconstitution, GitFlic
helixdevelopment/helixconstitution, GitVerse helixdevelopment/
HelixConstitution.

Приоритетный уровень: Helix-первичный — обязательный элемент управления, определяющий принципы построения всего семейства Helix.